

- [SumTube: How to Summarize Hours of Podcasts with a Local LLM](#)
 - [The Core Challenge: Context is King, and It's Limited](#)
 - [How It Works: The Summarization Pipeline](#)
 - [The Magic Ingredient: Hierarchical Prompt Engineering](#)
 - [1. Role Assignment](#)
 - [2. Instructional Scope](#)
 - [3. Hierarchical Architecture](#)
 - [Why This Approach Matters](#)
 - [Technical Highlights & Design Patterns](#)
 - [Getting Started: Configuration and Use](#)
 - [Prerequisites](#)
 - [Configuration](#)
 - [End-to-End Workflow](#)
 - [What This Project Teaches](#)

SumTube: How to Summarize Hours of Podcasts with a Local LLM

An in-depth look at building an AI pipeline that transforms lengthy YouTube podcasts into structured, professionally-written summaries, all running on your local machine.

sumtube-final-report-1.png

In an age of information overload, long-form podcasts are both a treasure trove of knowledge and a significant time commitment. A three-hour interview with a leading expert might contain invaluable insights, but finding the time to listen is a luxury not everyone has. What if you could capture the essence of that content in a fraction of the time?

This project, **SumTube**, is an automated pipeline that does just that. It fetches any podcast from YouTube, intelligently processes its transcript, and uses a locally-run Large Language Model (LLM) to generate a polished, well-structured summary in PDF format.

This document serves as both a user guide and a deep dive into the architecture, prompt engineering techniques, and design patterns that make this system possible. It's a practical demonstration of how to build sophisticated AI workflows that overcome the inherent limitations of today's models.

The Core Challenge: Context is King, and It's Limited

The biggest hurdle in summarizing long-form content is the finite **context window** of LLMs. A typical podcast can easily exceed 100,000 tokens (the words and sub-words the model processes), while many powerful open-source models are limited to context windows of 8,000, 16,000, or 32,000 tokens. You simply can't paste an entire transcript into the model and ask for a summary.

SumTube is designed around this constraint. Its core architecture is a multi-stage process built to "divide and conquer" the content.

How It Works: The Summarization Pipeline

At its heart, the class implements a multi-stage process:

1. **Extract Podcast Content:** The pipeline begins by pulling the video title, transcript, and thumbnail directly from YouTube.
2. **Chunk the Transcript:** The full transcript is broken down into smaller, overlapping text chunks. This ensures each piece fits comfortably within the LLM's context window while preserving continuity.
3. **Summarize Chunks Independently:** Each chunk is fed to the LLM with a specific prompt, asking it to create a detailed, objective summary.
4. **Generate Structured Report Sections:** The individual chunk summaries are then combined and given to the LLM in a series of "synthesis" steps to create a unified introduction, a structured body with topic-based headers, and a coherent conclusion.
5. **Compile and Polish:** A final prompt asks the LLM to assemble these sections into a single, polished document, applying consistent formatting (like APA style).
6. **Save to PDF:** The final markdown report is converted into a professional-looking PDF.

A key feature of this pipeline is its **checkpointing** system. Each major step is decorated with a `@checkpoint`, which saves the progress. This makes the process resilient to

interruptions—if you stop the script midway, you can restart it, and it will pick up right where it left off. It also allows for experimentation: you can tweak a prompt, change the model, and re-run the pipeline without having to re-process the initial steps.

The Magic Ingredient: Hierarchical Prompt Engineering

The quality of the final summary is entirely dependent on the quality of the instructions given to the LLM. SumTube doesn't use a single, massive prompt. Instead, it employs a **hierarchical prompting** strategy, guiding the model through a series of transformations from raw transcript to publishable content.

This pipeline demonstrates three advanced prompt engineering techniques:

1. Role Assignment

At each stage, the model is assigned the role of a specific expert: an "AI research assistant" for initial processing, a "professional summarizer" for the chunk-level work, and a "professional writer" for the final polish. Assigning a role consistently improves factual tone, reduces hallucinations, and keeps the writing style stable.

Example: Final Polishing Prompt

```
FINAL_REPORT_SYSTEM_PROMPT = (  
    "You are a professional writer..."  
    "You apply APA formatting..."  
)
```

This final-stage instruction focuses the model on refinement, ensuring voice alignment, academic style, and consistent formatting for a publication-ready document.

2. Instructional Scope

Each prompt is narrowly scoped to a single task. One prompt handles summarizing a single chunk, another integrates multiple summaries, and others are dedicated to creating

the introduction or conclusion. By telling the model exactly what to do—and what *not* to do—we prevent "prompt drift" and ensure each step performs its function precisely.

Example: Report Body Creation Prompt

```
CREATE_REPORT_BODY_PROMPT = (  
    "Integrate the provided ==SubContext== into a unified body..."  
    "Do NOT include introduction or conclusion..."  
    "Organize the material into ### topic headers..."  
)
```

This prompt explicitly forces a topic-based hierarchy, structured headings, and de-duplication of ideas, resulting in a well-organized and logical flow.

3. Hierarchical Architecture

The system uses a layered prompt architecture where the output of one stage becomes the input for the next.

Level	Purpose
System Prompts	Defines the model's overall tone, persona, and output format.
Instruction Prompts	Provides step-by-step guidance for a specific task.
Text Context	The actual transcript or summary content being processed.
Output	The structured result that feeds into the next stage.

This architecture allows the system to build understanding incrementally: raw chunks become detailed summaries, summaries are synthesized into thematic sections, and sections are assembled into a polished final report.

Example: Chunk Summarization Prompt

```
SUMMARIZE_CHUNK_PROMPT = (  
    "As a professional summarizer, create a detailed summary..."  
    "Use the == Title == ..."   
    "not exceeding {max_summary_response_size} bytes..."  
)
```

This initial summarization prompt is crucial. It enforces size constraints, demands an objective tone, and establishes a consistent format that makes the later synthesis

Why This Approach Matters

A naive summarization attempt would fail because podcasts exceed model context limits, dialogue is often unstructured, and transcripts are filled with noise. By applying segmentation, scoped prompting, role consistency, and hierarchical refinement, SumTube produces a factual, well-structured, and coherent document.

And it all runs locally with open-source models.

Technical Highlights & Design Patterns

The codebase demonstrates several powerful software engineering practices:

1. **Resilience with Checkpoints:** The `@checkpoint` decorator automatically saves the output of a function to a file. On subsequent runs, if the file exists, the function is skipped, and the result is loaded from the cache.
2. **Smart Resource Management:** The system intelligently calculates the maximum size for each chunk summary to ensure the combined summaries don't overflow the context window during the final synthesis step.
3. **Flexible Configuration:** A `config()` method allows for runtime customization of the model, temperature, context window size, and chunking parameters, making it easy to experiment with different settings.
4. **Performance Monitoring:** Built-in timing utilities track the duration of each major step, helping identify performance bottlenecks.

Getting Started: Configuration and Use

The entire process is orchestrated through a single `create_summary_report()` method.

Prerequisites

1. **Ollama:** You must have [Ollama](#) installed and running.
2. **LLM Model:** Pull a model to use for summarization. The default is `gemma2:9b`, but you can configure it to any model available in your Ollama library.

```
ollama pull gemma2:9b
```

3. **YouTube API Key:** To fetch transcripts, you'll need a YouTube API key. You can obtain one from the [Google Developers Console](#).

Configuration

The `config()` method allows you to customize key parameters:

- **model_name:** The name of the Ollama model to use (e.g., `'llama3'`).
- **temperature:** Controls creativity (0.0 for deterministic, 1.0+ for more creative).
- **num_cxt:** The context window size for the LLM. This cannot exceed the model's limit but can be lowered to save resources.
- **raw_text_chunk_size:** The size (in bytes) of each transcript chunk.
- **text_chunk_overlay_size:** The number of bytes to overlap between chunks to maintain context.

End-to-End Workflow

Here is a simplified view of the `create_summary_report` execution flow:

```
def create_summary_report(self):
    # 1. Get transcript and metadata
    self._get_title_and_transcript()

    # 2. Chunk the transcript
    chunks = self._chunk_transcript()

    # 3. Summarize each chunk individually
    self._summarize_chunks(chunks, max_summary_response_size)

    # 4. Concatenate the chunk summaries
    concatenated_content = self._read_and_concatenate_summaries()

    # 5. Generate structured report sections
    introduction_text = self._introduction_text(concatenated_content)
    main_body_text = self._main_body_text(concatenated_content)
```

```
conclusion_text = self._conclusion_text(concatenated_content)

# 6. Assemble the final report
final_report_text = self._final_report_text(draft_report)

# 7. Convert the report to PDF
self._markdown_to_pdf(final_report_text)
```

What This Project Teaches

By exploring SumTube, you'll gain a deeper understanding of what it takes to build practical, multi-step AI systems:

- **Prompts as Programmatic Instructions:** Learn to think of prompts as code that defines model behavior.
- **Staged Logic for LLM Pipelines:** See why complex tasks require breaking the problem down into sequential, manageable steps.
- **The Power of Constraints:** Understand how explicit instructions and constraints guide an LLM toward a desired outcome.
- **Iterative Refinement:** Recognize that professional results are rarely achieved in a single step but require multiple phases of processing and polishing.

This project is a blueprint for anyone looking to move beyond simple Q&A chatbots and build robust applications that leverage the full power of local Large Language Models.